Approach — Conversational SHL Assessment Recommender

Problem

Recruiters start vague ("I'm hiring a Java developer"). The service is a **stateless** FastAPI agent that, over a multi-turn dialogue, clarifies intent and returns a grounded shortlist of **1–10 SHL assessments** (`name` , catalog `url` , `test_type`). It is graded by an automated replay harness that plays the user with an LLM, under hard limits: **≤8 turns**, **30s/call**, exact response schema, and **no URL that isn't in the catalog**. Design is shaped by those limits.

Architecture — one LLM call per turn

Every request carries the full history (stateless). Per turn we make **exactly one LLM call** — a *router* that classifies intent and extracts normalized constraints as strict JSON; Python does everything deterministic (retrieval, URL lookup, templating, schema validation). Rejected multi-call agent loops: they blow the 30s + 8-turn budget and add nondeterminism.

```
history ─▶ router (1 LLM call, JSON) ─▶ intent + constraints + search_query
                                         │
  CLARIFY   RECOMMEND/REFINE       COMPARE       REFUSE
  ask       retrieve→assemble      grounded      canned
  recs=[]   →templated reply       prose, recs=[] refusal
                    │
      catalog post-filter (id→canonical name/url/type) ─▶ Pydantic ─▶ response
```

Key decisions: - **Un-inventable URLs.** The LLM returns *names/constraints*, never URLs. Python looks up the canonical URL by catalog id and drops any id not in the catalog. This makes hallucinated URLs *structurally impossible*, not just unlikely. - **Stateless refine is free.** History is replayed each call, so "refine" is just re-deriving all constraints and re-retrieving — we never hold state. - **Templated shortlist replies.** The schema-critical reply is built from a template, so the contract never depends on LLM formatting. CLARIFY/COMPARE prose (which needs fluency) is the only LLM-authored text. - **Robustness:** a ~25s request timeout, background model/index warmup so `/health` is instant, forced commit near the turn cap, defensive parsing of empty/garbage payloads, and a `SAFE_FALLBACK` that keeps every response valid.

Retrieval

Catalog = **370 Individual Test Solutions** parsed to `data/catalog.json` (Job Solutions excluded). Hybrid retrieval fuses two signals with **Reciprocal Rank Fusion**: - **Dense** — `all-MiniLM-L6-v2` , embeddings precomputed and shipped (`embeddings.npy`), so nothing large downloads cold. Catches semantic intent ("stakeholders" → interpersonal). - **BM25** (`rank_bm25`) over the same document text, with a subword tokenizer that splits letter/digit boundaries so exact ids resolve ("OPQ" → `opq32r` , ".NET" → `net`). Catches identifiers dense retrieval misses.

Metric is **Recall@10**, so we retrieve inclusively. The labeled shortlists are *batteries*, so after retrieval we **assemble**: role/skill spine + two recurring defaults the user rarely names — a personality measure (OPQ32r, `P` ; in 7/10 finals) and a cognitive measure (Verify G+, `A`) — added for hiring queries unless the user opts out (proximity-matched, so "keep Verify, drop OPQ" is honored).

Prompt design — router

System prompt encodes the conversation policy from the 10 traces: turn-1 vague → CLARIFY (never recommend); commit once **role + one discriminating attribute** is present; "no preference" → commit; **≤2 clarifiers**; REFUSE off-topic / legal / general-hiring / injection. Output is a fixed JSON shape (intent, constraints, `search_query` , `named_assessments` , `reply_text`); malformed output → safe CLARIFY. `search_query` is told to list *every* distinct skill so multi-skill JDs don't drop a technology.

Evaluation — real numbers

Two harnesses. `eval/recall_eval.py` is a **deterministic** single-shot proxy that maps each trace's labeled shortlist to catalog ids and measures retrieval Recall@10 — fast and reproducible, used for tuning. `eval/replay.py` runs each trace as a **real multi-turn conversation** through the agent (one router call per turn; scripted *or* LLM-simulated user), measuring **conversational** Recall@10 + groundedness and flagging any LLM-failure turns so the mean stays honest. `eval/probes.py` runs the behavior probes; `eval/report.py` aggregates all three into `eval/REPORT.md` . Per-change tuning results (`eval/TUNING_LOG.md`):

Change	Mean Recall@10
Baseline hybrid dense+BM25+RRF	**0.5012**
+ RRF constant / N sweep	~0.50 (flat)
+ enriched embedded text	0.6931 (flat)

Change	Mean Recall@10
+ test_type-aware battery assembly	**0.6931 (+0.19)**
+ router-synthesized query	0.7131
+ "every distinct skill" query prompt	**~0.72**

Behavior probes (binary, all passing via the test suite): refuses off-topic/ injection, no turn-1 recommend on a vague opener, honors edits/refine, 100% grounded. 118 unit/integration tests cover schema, retrieval, assembly, agent paths, and endpoint robustness (timeout/fallback).

What didn't work

- **RRF/N tuning** and **text enrichment** were flat — the misses are missing *documents*, not mis-ranked ones, and the catalog has no competencies field to add.
- **Per-skill / per-message multi-query fusion** made recall *worse* (0.486): sub-queries pulled unrelated items and diluted the exact-name signal.
- Remaining misses are near-duplicate product families (short variant loses to `-365` sibling) and skill-flooded JDs — ranking/diversity problems, not fusion tuning. Returns flattened, so tuning stopped at ~0.72.

Limitations (named honestly)

- **Overfit risk:** the +0.19 default-injection was tuned on the 10 *public* traces; OPQ32r/Verify G+ may be less dominant on the holdout set, so the number may not fully transfer. It's the best signal available, but it's a fit to 10 conversations.
- The conversational replay caught a real bug the single-shot proxy hid — a refine/confirm turn retrieving only from the sparse last message — now fixed by combining the router query with the full history (C9: 0.29 → 0.86 conversational recall). This is why the multi-turn harness matters, not just the proxy.

AI tools used

Built with **Claude Code** (Anthropic) for implementation, retrieval tuning, and evaluation. Runtime LLM: **Groq** `llama-3.3-70b-versatile` (chosen for latency under the 30s cap), behind an `app/llm` interface with a Gemini swap available by changing one env var.